CS 598 Research Project Report: Group-7

Rohan Pareek
University of Illinois at Urbana-Champaign
United States
rpareek3@illinois.edu

Jamie Fulford
University of Illinois at Urbana-Champaign
United States
jamiehf2@illinois.edu

Abstract

More and more developers are writing software solely through vibe coding, building entire programs by chatting with a coding agent. Over a long session, the developer will place many constraints upon the code that the agent then agrees upon; each new request can break one that they agreed to earlier. Vibe coders rarely read the generated code, so these broken constraints may go unnoticed. How can we gain confidence that vibe-coded software actually does what the developer asked for? Historically, formal verification has offered such guarantees that regular software testing cannot, and recently LLMs have made formal verification much more accessible. However, current approaches either audit code offline or merely fix a specification before any code is written—none of them track constraints during a session. We propose a Lean companion for vibe coding that does exactly this. Our companion listens to the developer-agent conversation, formalizes the agreed-upon constraints as Lean invariants, and builds a model of the code in Lean. Whenever the coding agent makes an edit, the companion checks every short sequence of actions in the model against these invariants and returns failed checks to the agent, thus catching broken constraints before the developer has to.

1 Artifact Information

Link to the GitHub Repository: https://github.com/Rohpar1504/CS-598-Research-Project-Group-7

2 Introduction

Large language models are being rapidly deployed as fully autonomous agents. As these agents' capabilities have grown over the past few years, more developers are using this autonomy to rapidly prototype code via vibe coding—writing software completely by chatting with an agent. While vibe coding lets developers generate more code than ever before, the iterative back-and-forth of the conversation places many constraints on the generated code. In particular, these constraints might disagree with each other as they evolve over time, resulting in significant technical debt for the agent, and more importantly, the developer. Worse yet, LLM-generated code has been known to contain security flaws [4], and since the developer is vibe coding, they are less likely to notice flaws. This raises the question: how can we gain confidence over vibe-coded software?

Formal verification is one such way we can acquire guarantees on the properties of a program, and it offers a unique certainty of correctness that simple testing cannot. Tools such as proof assistants (e.g. Lean [3], Rocq [16]) and specification languages (e.g. TLA+ [10]) are often used for formal verification, but have historically required too much expertise and development time to justify its use. LLMs have significantly lowered both costs. For example, Specula [1] uses agents to write TLA+ models of large systems and check them for bugs, and Quint provides tooling for agents to write executable specifications from requirements or source code [14].

None of the existing techniques track constraints during a vibe-coding session. Current approaches either are purely offline audits, or specification-driven tools that merely fix requirements before any of the code generation begins. Mitchell and Shaaban [11] propose a formal side-car that would track these constraints, but their prototype only performs lightweight syntactic checks.

We propose a Lean companion that runs in the background during a vibe-coding session. The companion will listen to the developer-agent conversation and formalize the constraints the developer and agent agree upon as Lean invariants, as well as maintain a Lean model of the generated code for checking. This process repeats each time the agent generates new code.

We evaluate the companion on the stateful problems of SlopCodeBench [12]. We run a coding agent with a simulated developer and compare the companion against property-based tests generated from the same invariants. We then measure any violations that are detected, how accurately the companion extracts invariants, and the overhead it adds.

3 Proposed Technique

3.1 Approach Overview

Our design is a companion for vibe coding. The companion runs alongside a coding agent during a session while maintaining a model of the application's state in Lean. It checks this model against invariants formalized from the conversation and runs after every edit the coding agent makes. The companion operates in four stages (Figure 1):

(1) *Constraint Extraction.* The companion translates constraints stated by the developer into Lean invariants. One example of such a constraint is "There should be no double-booking of a room". These invariants are confirmed by the developer through examples.
(2) *Lean Model.* An LLM updates a Lean model of how the code's actions change the application's state. Conformance replay runs the same action sequences on the code and the model, and repairs the model if the states differ.
(3) *Invariant Checking.* The companion exhaustively checks every short action sequence in the model against the invariants.
(4) *Feedback.* If we find a sequence that violates an invariant, then we translate that sequence into a test on the code. If that test fails, then the result is returned to the coding agent.

4 Experimental Evaluation

We organize the evaluation around four research questions:

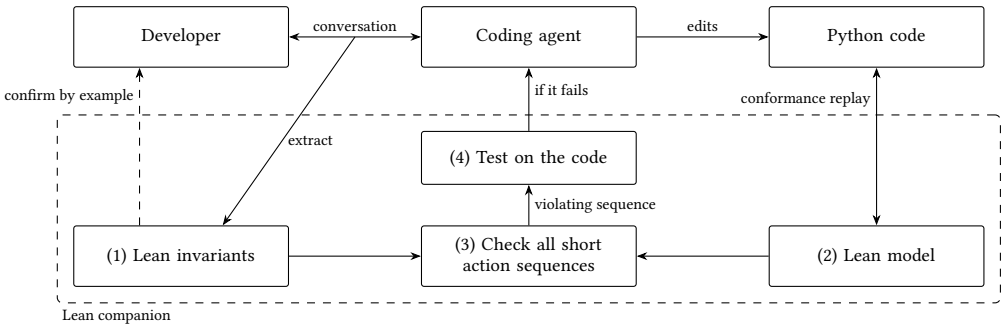


Figure 1: Overview of the Lean companion.

RQ1 (Formalization fidelity) How accurately does the companion translate constraints stated in the developer-agent conversation into Lean invariants, and how reliably does it update or retire constraints that are later revised or contradicted?

RQ2 (Violation detection) How effectively does the companion detect generated code that violates previously agreed-upon constraints, compared with property-based tests generated from the same invariants?

RQ3 (Regression mitigation) When detected violations are returned to the coding agent, does the companion reduce regression accumulation over a multi-turn session relative to an unassisted agent and a test-based verification loop?

RQ4 (Overhead) What latency, token, and failure overhead does the companion introduce, and how does this overhead scale as the session and the generated code grow?

4.1 Evaluation Setting

Benchmarks. Our primary benchmark is the stateful subset of SlopCodeBench [12], whose Python problems evolve over a sequence of checkpoints and rerun the tests from all earlier checkpoints at each one as a regression oracle. Unlike function-level suites [6], these problems maintain persistent state governed by rules, such as a configuration service with approval workflows. As a secondary benchmark, we use a subset of EvoCode-Bench [15], whose longer tasks include corrections and conflicts that supersede earlier requirements.

Simulated developer. An LLM-based simulated developer delivers each checkpoint's specification change conversationally, paraphrasing it, splitting it across messages, or leaving it implicit. It also answers the companion's confirmation questions according to the specification. The checkpoint specifications are fixed across configurations and serve as ground truth.

Configurations. All configurations use Claude Code as the coding agent and share a one-retry budget per checkpoint: (1) agent-only, with no side-car; (2) an oracle gate, modeled on the Verification Gate of Huang et al. [6], which reruns the oracle and rolls back on regression, and which serves as an upper bound since it has access to ground truth; (3) a PBT side-car, which translates our extracted invariants into property-based tests [7]; and (4) our Lean companion, which checks the invariants against an extracted Lean model and returns failures to the agent. The PBT side-car and the Lean companion receive the same time budget per check.

Metrics. For RQ1, we measure precision and recall of generated invariants against the constraints in the checkpoint specifications, with and without developer confirmation, and the fraction that typecheck in Lean. For RQ2, we measure detection recall, precision, and turns-to-detection, using oracle failures as ground truth and manually inspecting alarms the oracle misses. For RQ3, following Huang et al. [6], we report the per-checkpoint oracle pass rate, the degradation from the first to the final checkpoint, and the fraction of problems that regress. For RQ4, we measure added latency, tokens, the rate of failed or timed-out checks, and the rate of model drift detected by conformance replay.

5 Related Work

Our work builds on three areas of research: formal side-cars for vibe coding, model extraction and conformance checking, and intent formalization.

Formal Side-cars. Mitchell and Shaaban [11] argue for "Vibe Reasoning" to go along with vibe coding in their paper; in particular, they argue that vibe coding can suffer from a breakdown in the reconciling of user constraints over a long coding session. To this end, they propose a side-car that automatically formalizes specifications from the conversation and validates the code against them. If there is a failure in the check, then this is returned to the coding agent. Our system is an implementation of this side-car with a semantic model and exhaustive checking. Similarly, VibeContract [17] breaks down the intent of the natural language into specific tasks with generated contracts. These contracts then guide the validation and debugging of the program. Forge [18] is a closed-loop pipeline that extracts models from source code and checks them with a verifier. However, this pipeline only works with a restricted subset of Java so that they can extract their models.

Model Extraction and Conformance. Specula [1] is a tool that generates formal specifications in TLA+ for large code repositories, and validates the specifications against execution traces. However, it only runs offline, takes hours per repository, and importantly generates invariants mostly from implemented code, so that a constraint the code already violates may never be translated into an invariant. We use the same method of validating from execution

traces. Quint [13] is an executable specification language based on TLA that is equipped with a model checker. Importantly, its tooling allows agents to generate Quint specifications from requirements or given source code [14]. Developers can invoke these tools on demand, while our companion maintains its specification model automatically during a vibe coding session. One tool built on Quint is FuzzMo [2]. FuzzMo uses an LLM to synthesize Quint models from code and then checks conformance by comparing Quint execution traces against the implemented code. The authors distinguish code-based models, which are specifications that match the implementation, from ideal-based models, which are specifications that match what the software should do. Our design is a hybrid of these two approaches; our specification model follows the code while the invariants that the model is checked against come from the conversation.

Intent Formalization. Much of our approach relies on recognizing the intent of a user throughout a vibe coding session. Lahiri [8] argues that translating this intent into formal specifications is an important challenge for LLM-generated code, and that only the user can judge the correctness of such specifications. We address one instance of this by extracting invariants from the vibe coding conversation. TiCoder [9] uses user feedback on generated tests to formalize intent. Similarly, we confirm extracted invariants with the developer. nl2postcond [5] studies using LLMs to translate code documentation into postconditions. The authors evaluate the postconditions' correctness and how well they distinguish incorrect code. However, this only targets individual functions, not an entire program's state. In industry there are specification-driven tools such as Kiro [7], which prompt an agent to write a specification before writing any code. Kiro translates the properties from its specification into property-based tests. Still, this tool does not check these specifications against a formal model.

References

[1] Qian Cheng, Saad Mohammad Rafid Pial, Ruize Tang, Yiming Su, Emilie Ma, Finn Hackett, Ivan Beschastnikh, Yu Huang, and Tianyin Xu. 2026. Specula: Scaling Formal Specifications for Autonomous Model Checking of System Code. arXiv:2607.25333 [cs.SE] https://arxiv.org/abs/2607.25333

[2] Jan Corazza, Ivan Gavran, Gabriela Moreira, and Daniel Neider. 2025. Accessible Smart Contracts Verification: Synthesizing Formal Models with Tamed LLMs. arXiv:2501.12972 [cs.SE] https://arxiv.org/abs/2501.12972

[3] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37

[4] Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, and Yizheng Chen. 2025. SecRepoBench: Benchmarking LLMs for Secure Code Generation in Real-World Repositories. arXiv:2504.21205 [cs.CR] https://arxiv.org/abs/2504.21205

[5] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024). arXiv:2310.01831. doi:10.1145/3660791

[6] Yonghui Huang, Lin Ma, Amjed Tahir, Qian Zhang, Liwen Xiao, and Lysa Xiao. 2026. Regression Accumulation in Multi-Turn LLM Programming Conversations. arXiv:2607.01855 [cs.SE] https://arxiv.org/abs/2607.01855

[7] Kiro. 2025. Does Your Code Match Your Spec? Property-Based Testing in Kiro. https://kiro.dev/blog/property-based-testing/. Accessed September 2026.

[8] Shuvendu K. Lahiri. 2026. Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents. arXiv:2603.17150 [cs.SE] https://arxiv.org/abs/2603.17150

[9] Shuvendu K. Lahiri, Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, Madanlal Musuvathi, Piali Choudhury, Curtis von Veh, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2022. Interactive Code Generation via Test-Driven User-Intent Formalization. arXiv:2208.05950 [cs.SE] https://arxiv.org/abs/2208.05950

[10] Leslie Lamport. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

[11] Jacqueline Mitchell and Yasser Shaaban. 2025. Position: Vibe Coding Needs Vibe Reasoning: Improving Vibe Coding with Formal Verification. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages (LMPL '25)*. ACM. arXiv:2511.00202. doi:10.1145/3759425.3763390

[12] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. 2026. Slop-CodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks. arXiv:2603.24755 https://arxiv.org/abs/2603.24755

[13] Quint Contributors. 2026. Quint: An Executable Specification Language Based on the Temporal Logic of Actions. https://github.com/quint-co/quint. Accessed September 2026.

[14] Quint Contributors. 2026. Quint LLM Kit: Agents and Tools for Using Quint with LLMs. https://github.com/quint-co/quint-llm-kit. Accessed September 2026.

[15] Haiyang Shen, Xuanzhong Chen, Wendong Xu, Yun Ma, Liang Chen, and Kuan Li. 2026. EvoCode-Bench: Evaluating Coding Agents in Multi-Turn Iterative Interactions. arXiv:2605.24110 [cs.SE] https://arxiv.org/abs/2605.24110

[16] The Rocq Development Team. 2025. The Rocq Prover. https://rocq-prover.org.

[17] Song Wang. 2026. VibeContract: The Missing Quality Assurance Piece in Vibe Coding. arXiv:2603.15691 [cs.SE] https://arxiv.org/abs/2603.15691

[18] Ran Wei, Le Zhu, Haochi Wang, Jim Woodcock, Fang Yan, Simon Foster, and Xiangyang Ji. 2026. Formal-Method-Guided Vibe Coding: Closing the Verification Loop on AI-Generated Safety-Critical Software Through Model-Driven Engineering. arXiv:2606.22413 [cs.SE] https://arxiv.org/abs/2606.22413